PROJECTS UNDERGONE

Prepared by: Pranav Vivek Sadwelkar

1. AIDRAC Agentic AI Disaster Response Coordinator

HACKATHON PROJECT

Project Type	Major Project (Full-Stack, Agentic AI System)
Domain	Disaster Management / Public Safety / Applied AI
Repository	https://github.com/pranav-1205/AIDRAC-Agentic-AI-Disaster-Response-Coordinator
Sponsorship	Lenovo Leap Hackathon Project

Abstract

AIDRAC is a full-stack disaster management platform that assists citizens during natural disasters by surfacing safe evacuation routes, nearby shelters, hospitals, police stations, fire stations, and pharmacies, alongside live weather data, government emergency alerts, and AI-driven decision support. The system combines a React/TypeScript frontend with a FastAPI backend, uses live OpenStreetMap and Overpass API data for infrastructure, and integrates a Gemini-powered multi-agent (LangGraph) system that reasons over weather, alerts, infrastructure, and routing to recommend the safest course of action.

Problem Statement

During natural disasters, citizens often lack a single, reliable, real-time source for safe evacuation routes, nearby emergency infrastructure, and official alerts. Existing tools are fragmented weather, government alerts, and location-based services are rarely combined with intelligent, context-aware decision support, leaving people to make critical safety decisions with incomplete information.

Objective / Solution

Build a unified web application that ingests live weather, government CAP alerts (IMD/NDMA), and OpenStreetMap infrastructure data, then uses a coordinator of specialised AI agents (Weather, Alert, Infrastructure, Route) to generate structured, risk-ranked recommendations including the nearest safe destination, reasoning, and an action checklist with a deterministic fallback when the AI service is unavailable.

Key Features

- JWT-based authentication with token session management
- Interactive Leaflet map with disaster zones, CAP alert polygons, and layered POI markers
- Live Overpass API queries for shelters, hospitals, police, fire stations, pharmacies, schools and community centres
- Emergency SOS button routing to the nearest safe facility
- Turn-by-turn walking route generation (OpenRouteService / OSRM) with Haversine fallback
- Weighted safe-destination scoring algorithm
- Live weather via OpenWeatherMap with auto-refresh
- Government CAP alert ingestion (IMD/NDMA) with geofencing and background polling
- Gemini-powered multi-agent AI Decision Assistant with deterministic fallback and incident memory
- Theme/accent customisation, admin dashboard, and PostgreSQL persistence

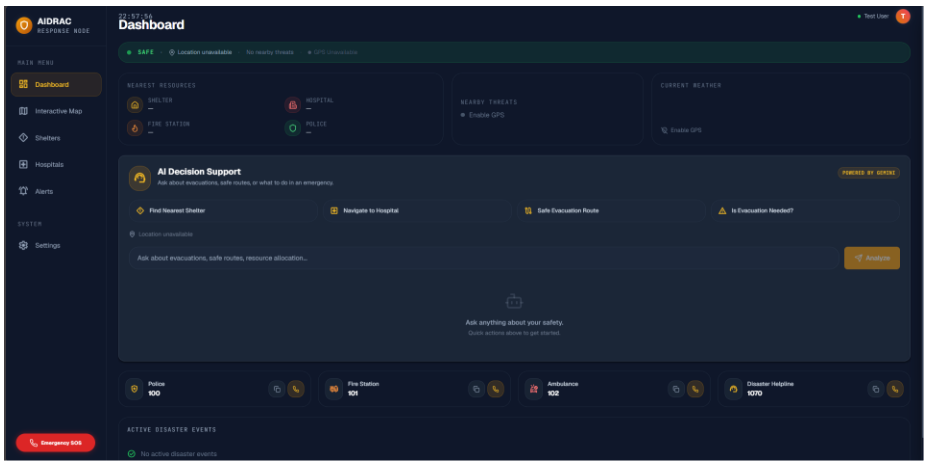
Technology Stack

Technology	Purpose
React + TypeScript, Vite, Tailwind CSS	Frontend UI, build tooling, and styling
Leaflet + react-leaflet	Interactive map rendering
Python, FastAPI, SQLAlchemy, Pydantic	Async backend API and data validation
LangGraph + google-genai (Gemini)	Multi-agent orchestration for AI decision support
PostgreSQL	Primary relational data store
JWT (python-jose) + bcrypt	Authentication and password security
Overpass API, OpenStreetMap, OpenRouteService/OSRM, OpenWeatherMap	External geospatial, routing, and weather data

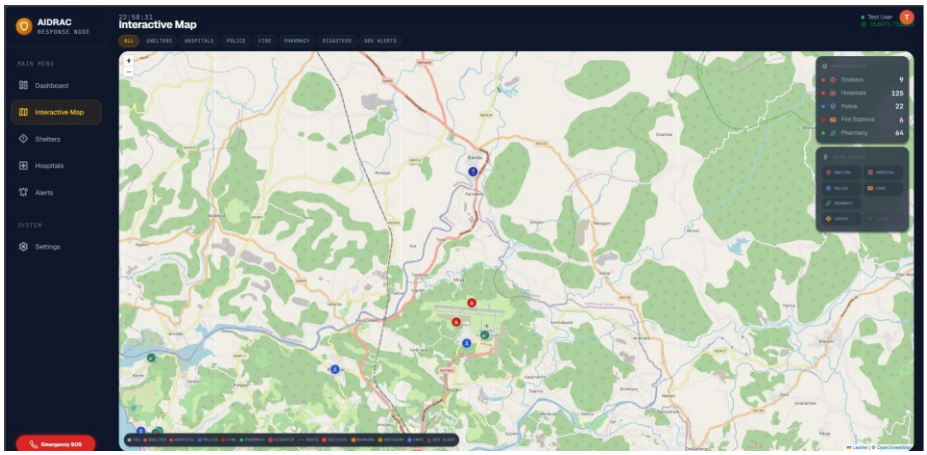
Outcome

A working full-stack prototype demonstrating agentic AI coordination for disaster response, with live government alert ingestion, real infrastructure data, and AI-assisted recommendations, deployed via Docker Compose.

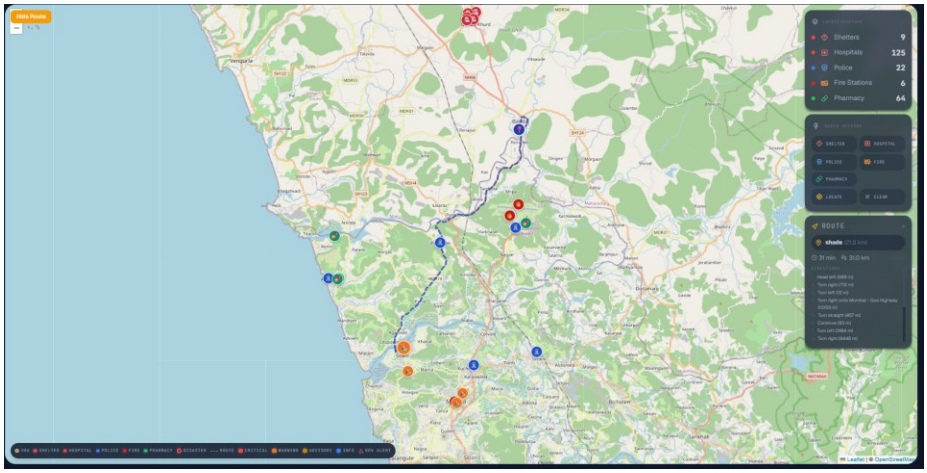
Screenshots



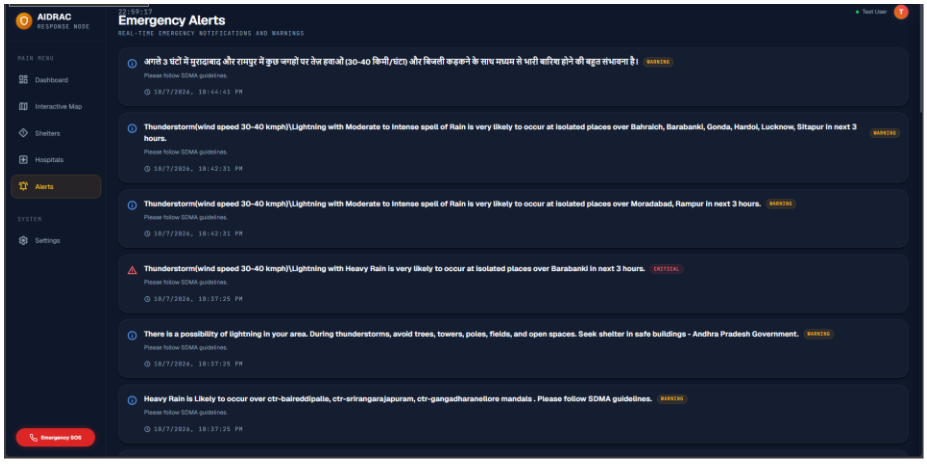
Dashboard - nearest resources, AI Decision Support, and emergency contacts



Interactive Map live infrastructure layers (shelters, hospitals, police, fire, pharmacy)



Interactive Map turn-by-turn safe route with infrastructure counts



Emergency Alerts real-time government (SDMA) notifications

2. ColdGuard Temperature Monitoring & Cooling System Simulation

HACKATHON PROJECT

Project Type	Hackathon Project (Full-Stack IoT Simulation System)
Domain	IoT / Cold-Chain Monitoring / Real-Time Systems
Repository	https://github.com/pranav-1205/ColdGuard
Sponsorship	Quantexera Hackathon *(Podium finish with 3rd rank)*

Abstract

ColdGuard is a temperature monitoring and cooling system simulation platform built with a React frontend and an Express.js backend. It models a cold-chain asset (e.g., a refrigerated transport unit) in real time, streaming temperature, battery, solar, GPS, and ESP32 sensor-signal data to a live dashboard over Socket.IO, while persisting history and events to MySQL for later review.

Problem Statement

Cold-chain and temperature-sensitive assets (perishables, medical supplies, sensitive equipment) require continuous monitoring to prevent spoilage or damage. Without a real-time, centralised system, threshold breaches, power/battery issues, or location deviations can go unnoticed until it is too late, and there is no simulated environment to test alerting logic before deploying real sensors.

Objective / Solution

Provide a simulation-driven monitoring engine that models temperature, battery, and solar behaviour over time, ingests external/IoT sensor packets (via REST and MQTT), relays live GPS and ESP32 signal data, raises threshold alerts, and visualises everything on a real-time dashboard giving developers and operators a safe environment to validate cold-chain monitoring logic end-to-end.

Key Features

- Real-time simulation engine (temperature, battery, solar, route) ticking on a fixed interval
- Live dashboard updates via Socket.IO (esp32_signals, gps_update, threshold_alert events)
- External sensor ingestion endpoint and MQTT topic subscription for IoT integration
- ESP32 signal processing service and GPS tracking service, each with their own event streams
- Historical simulation data, sensor readings, and event logs persisted to MySQL
- Map visualisation via an OpenStreetMap embed
- REST API for simulation state, history, events, and control (start/stop/tune parameters)

Technology Stack

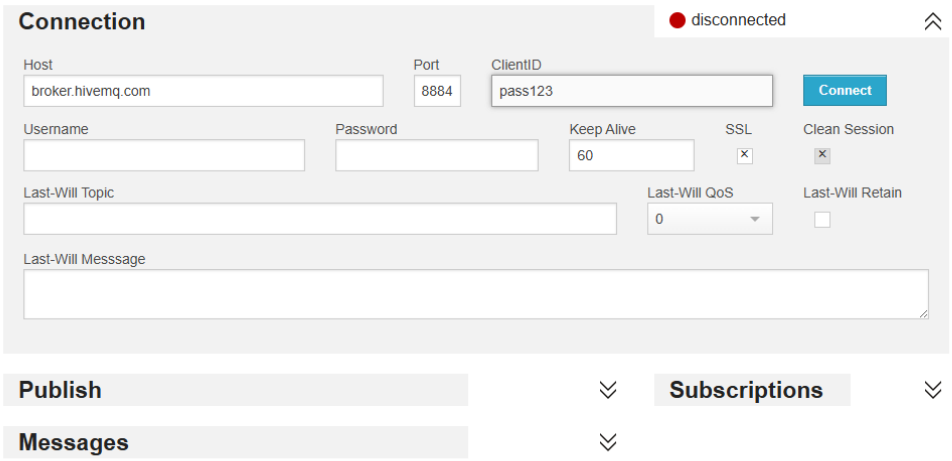
Technology	Purpose
React 19 + Vite	Real-time monitoring dashboard
Socket.IO (client + server)	Live bidirectional data streaming
Express.js 5, Node.js	Backend API and simulation engine

Technology	Purpose
MySQL (mysql2)	Persistent storage for readings, history, and events
MQTT	IoT sensor/topic ingestion
OpenStreetMap	Map-based location visualisation

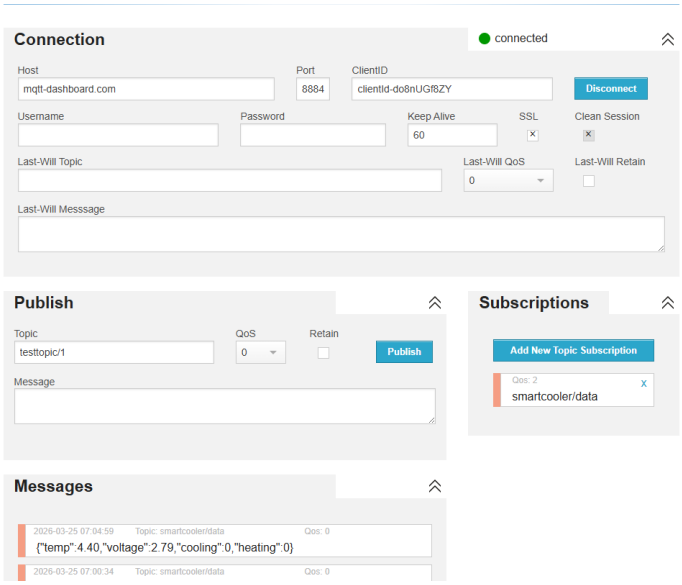
Outcome

A functioning simulation-and-monitoring pipeline that mirrors a real IoT cold-chain deployment, with real-time alerting, and a persisted history suitable as a foundation for connecting real hardware sensors.

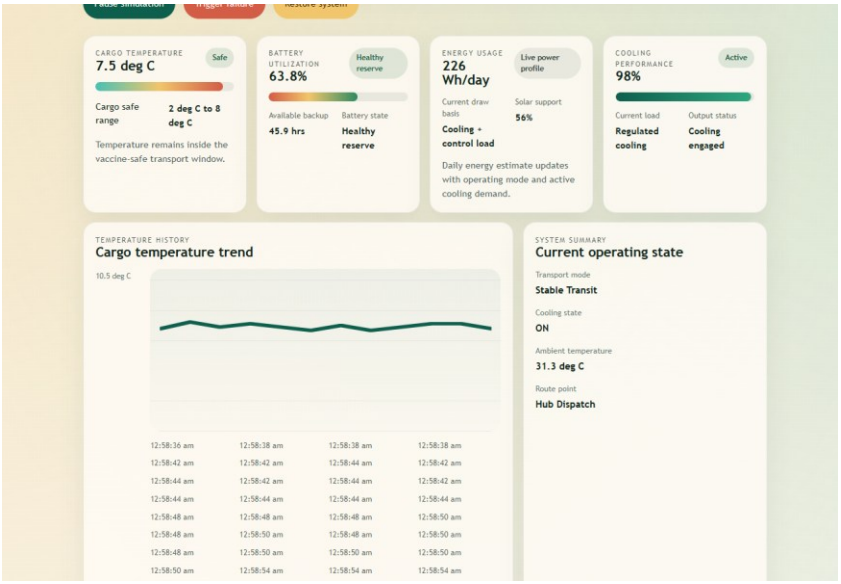
Screenshots



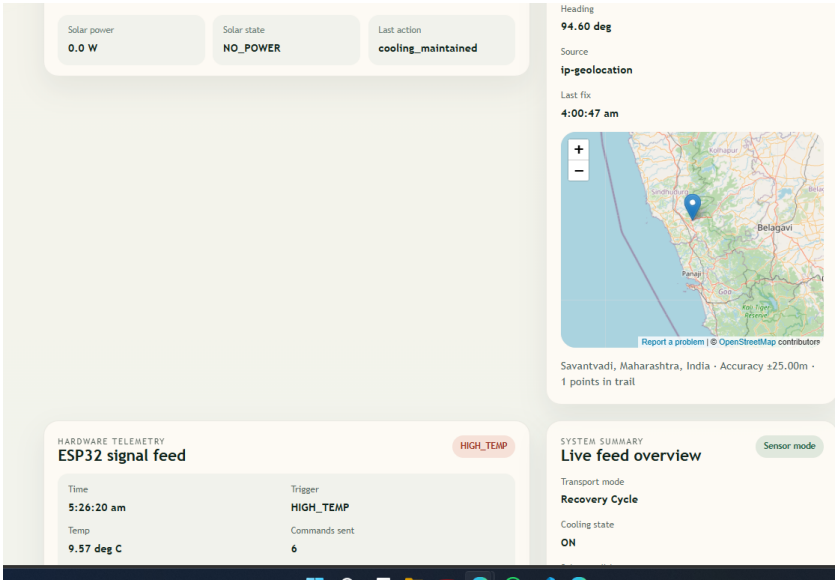
MQTT dashboard - connection configuration



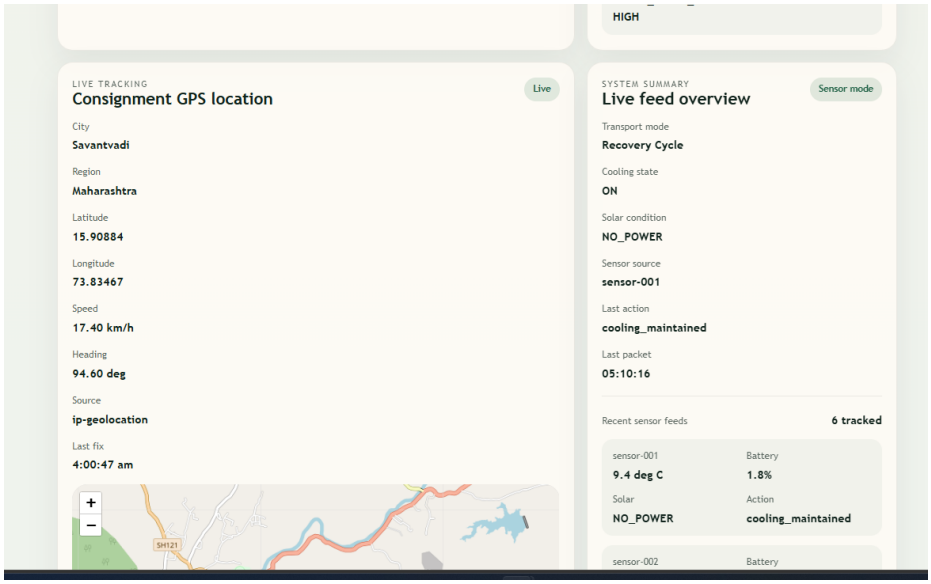
MQTT dashboard - connected and streaming live sensor data



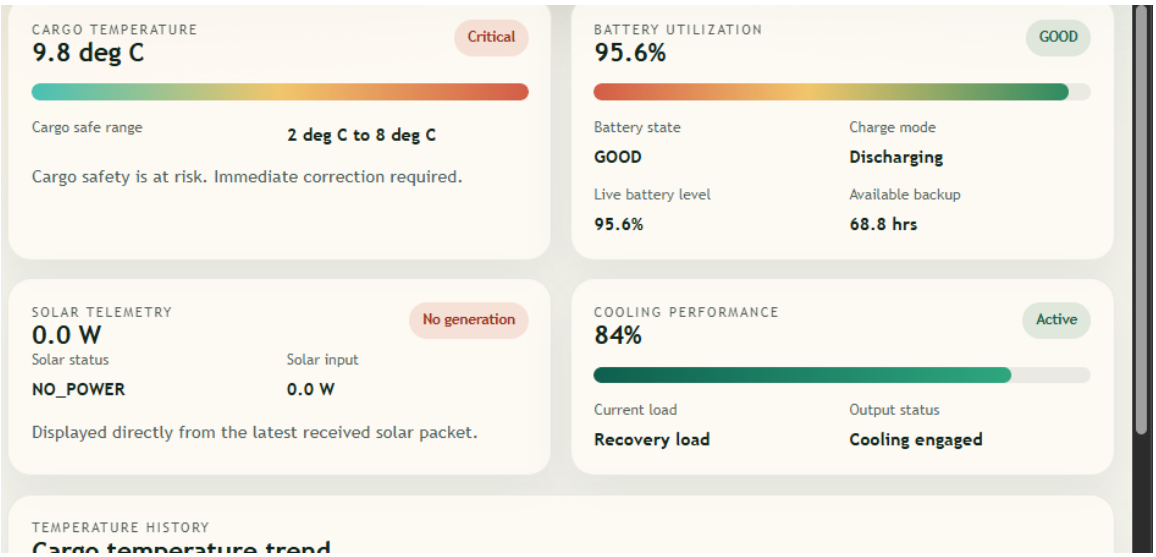
Wokwi ESP32 + DHT22 hardware simulation with sensor payloads



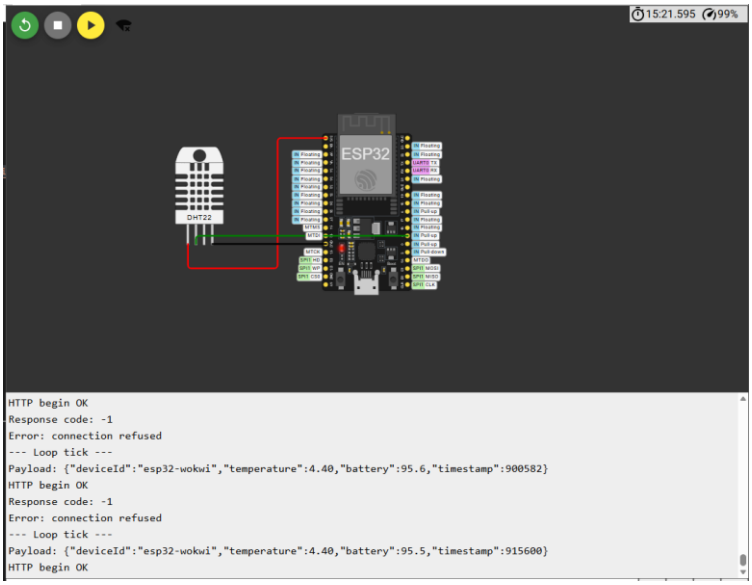
Cargo temperature, battery, solar and cooling performance metrics



Cargo temperature trend and current operating state



Live GPS tracking - consignment location details



Solar telemetry, heading and live GPS map

3. AI Study Buddy

MINI PROJECT

Project Type	Mini Project (AI-Powered Educational Tool)
Domain	EdTech / Applied AI (LLM-based)
Repository	https://github.com/pranav-1205/Study_Buddy
Sponsorship	Self-initiated / Independent project no external sponsorship
SDG Alignment	SDG 4 Quality Education
Author	Pranav Vivek Sadwelkar

Abstract

AI Study Buddy is an AI-powered educational assistant that lets students upload their own notes (PDF/TXT) and instantly interact with the material generating quizzes, explanations, and summaries. Built with Streamlit and powered by Groq API using Meta Llama models, it turns static study material into an interactive, personalised learning companion.

Problem Statement

Students often struggle when studying from long, dense notes or textbooks. Traditional revision methods lack interactive self-assessment, making it difficult to gauge understanding, and many learners lack access to personalised tutors or customised learning material slowing learning and reducing retention.

Objective / Solution

AI Study Buddy acts as a personalised, always-available study companion: students upload notes and the system generates tailored quizzes, evaluates answers instantly, explains difficult topics, and produces concise summaries with an additional Teacher Mode for educators to generate learning material and answer keys.

Key Features

- Upload PDF or TXT study material
- Interactive quiz generation with automatic evaluation and instant feedback
- AI-driven topic explanations and key-point summaries
- Simplified notes for difficult concepts
- Teacher Mode for generating learning material and answer keys
- Printable quiz/answer-key PDF export

Technology Stack

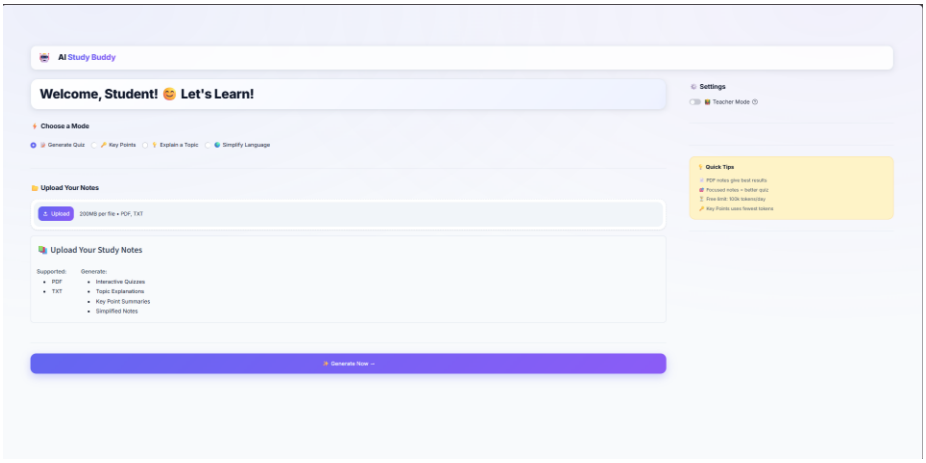
Technology	Purpose
Python, Streamlit	Core application logic and web interface
Groq API + LLaMA models	High-speed LLM inference for quizzes, summaries, and explanations
PyPDF2	Extracting text from uploaded PDF notes

Technology	Purpose
ReportLab	Generating downloadable PDF quizzes and answer keys

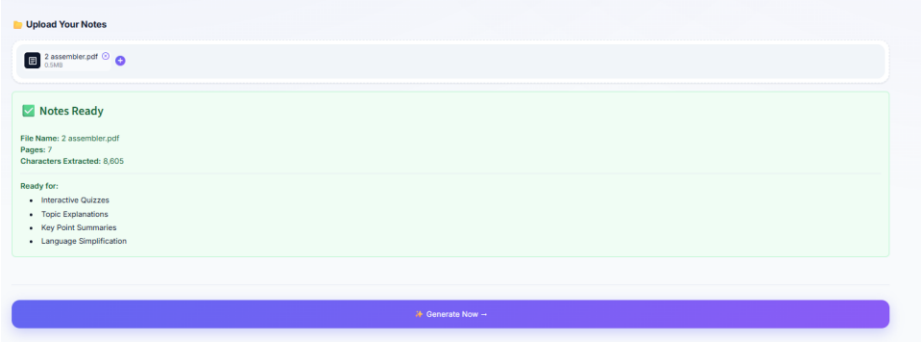
Outcome

A deployed Streamlit application giving students an interactive way to revise from their own notes, aligned with the goal of accessible, technology-driven education (SDG 4).

Screenshots



Homepage



Notes upload

4. Note WebApp - NoteMania

(College Mini Project - Yashwantrao Bhonsale Institute of Technology)

MINI PROJECT

Project Type	Mini Project (College Full-Stack CRUD Web Application)
Domain	Productivity / Personal Note-Taking
Repository	https://github.com/pranav-1205/Note_WebApp
Guide	Mr. P. H. Dongare
Sponsorship	Academic mini project - no external sponsorship

Abstract

NoteMania (Note WebApp) is a full-stack web application that lets users create, manage, and organise personal notes online. It provides a clean, intuitive interface with user authentication, real-time note management, and persistent cloud storage, so notes can be accessed securely from any device.

Objectives

- To provide user registration and login with JWT-based authentication
- To enable users to create, read, update, and delete (CRUD) notes
- To store notes persistently using a MongoDB cloud database
- To build a responsive frontend with React.js for cross-device usability
- To implement secure RESTful APIs using Node.js and Express.js

Problem Statement

In today's fast-paced digital world, individuals often struggle to manage their personal notes efficiently. Traditional note-taking methods (paper notebooks, local text files) lack accessibility across multiple devices and do not offer cloud synchronisation or secure storage.

Limited Accessibility: Without a web-based note management system, users are unable to access their notes from different devices, leading to a lack of productivity and difficulty retrieving important information at the right time.

Security Concerns: Notes stored locally are vulnerable to data loss, and most free note apps lack proper authentication. A secure, cloud-based note web app addresses these challenges directly.

Key Features

- User registration and login with hashed passwords (bcrypt) and JWT-based sessions
- Create, fetch, update, and delete personal notes
- Tagging of notes for organisation (defaults to "General")
- Calendar view to browse notes by date
- Trash bin with restore / permanent delete
- Archive for storing notes separately from the main dashboard
- Input validation via express-validator
- Responsive React UI built with Vite and Tailwind CSS

Technology Stack

Technology	Purpose
React.js, HTML5, CSS3	Frontend UI and client-side routing
Node.js, Express.js	Backend REST API
MongoDB (Atlas)	Cloud data persistence for users and notes
JSON Web Tokens (JWT) + bcrypt	Authentication and password security
JavaScript (ES6+)	Core application language
VS Code	Development IDE

Advantages

- Cloud Accessibility - notes are stored in the cloud and accessible from any device with a browser and internet connection
- Secure Authentication - JWT-based login ensures only authorised users can access their own notes
- Real-Time Updates - the React frontend gives instant feedback when notes are added, edited, or deleted, without page reloads
- Simple and Intuitive UI - a clean, minimal interface with no learning curve

Limitations

- Internet Dependency - requires an active internet connection; unavailable offline
- No Rich Text Support - current version supports plain text notes only (no tables)
- Limited Scalability the free-tier MongoDB Atlas cluster may face performance issues with many concurrent users

Outcome

Outcome
A fully functional MERN-stack note management application featuring secure authentication, cloud-based note storage, and complete CRUD functionality. The project demonstrates practical implementation of RESTful APIs, JWT authentication, MongoDB integration, and responsive React-based web development.

Implementation Screenshots

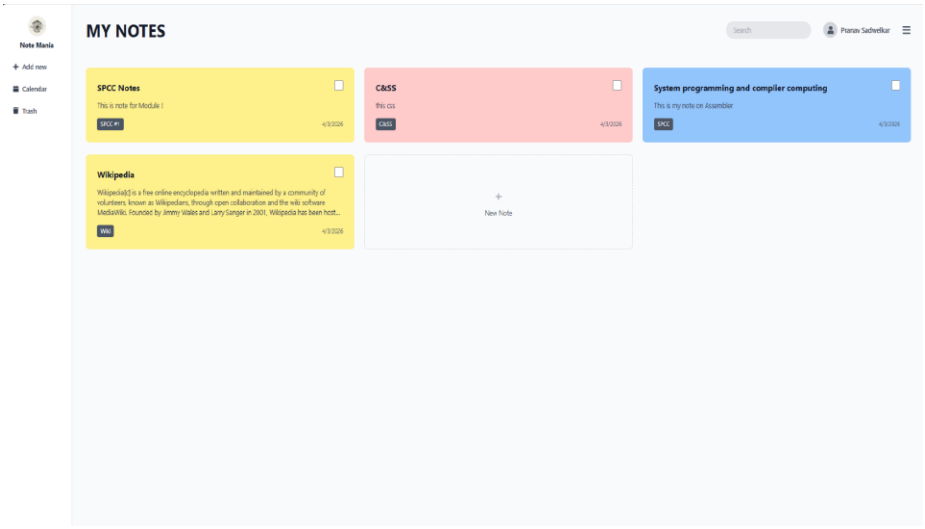


Fig: Dashboard My Notes

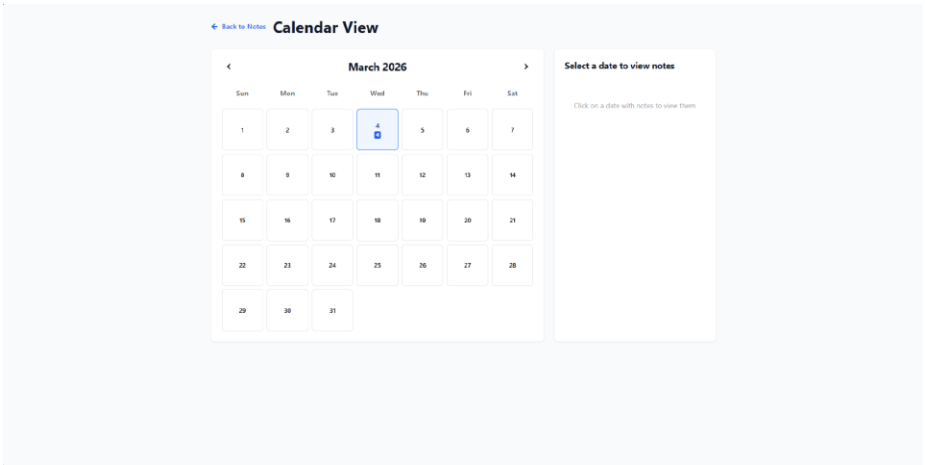


Fig: Calendar View

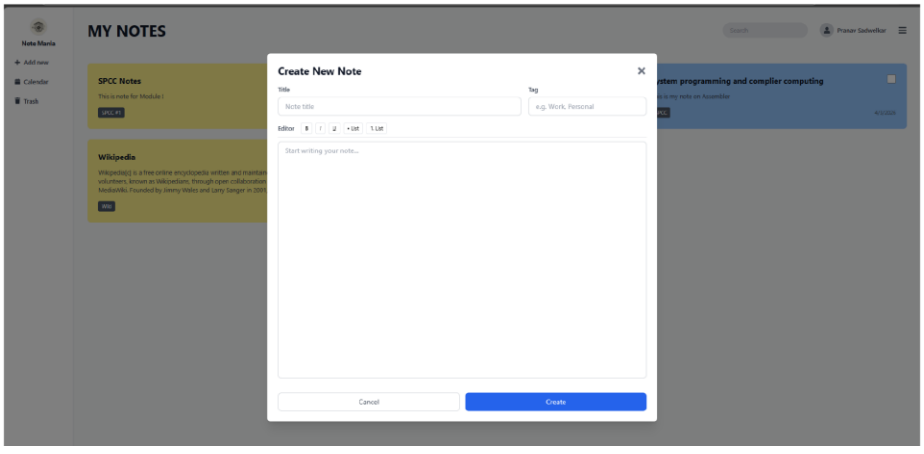


Fig: New Note Creation

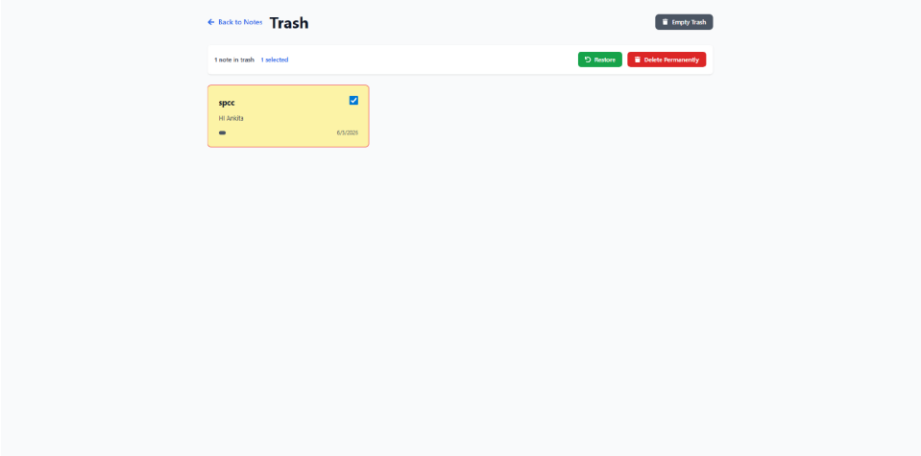


Fig: Trash Bin

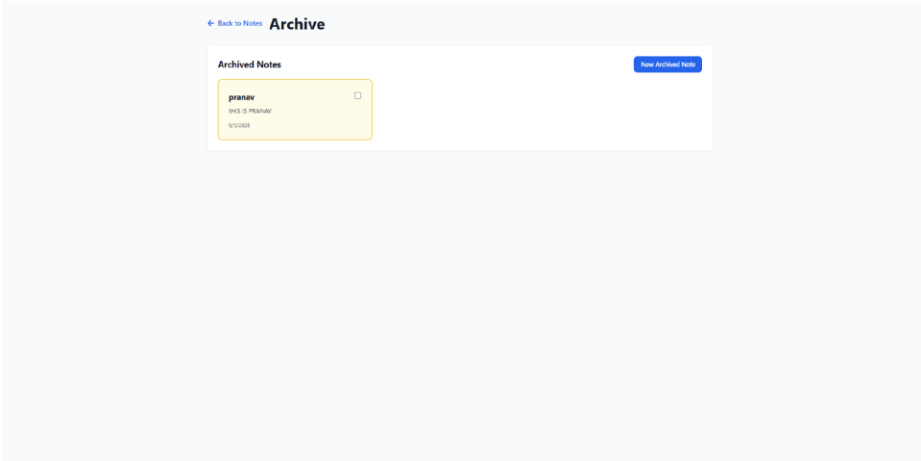


Fig: Archive

5. Penetration Testing Framework API

(Third-Year CSE Team Mini Project)

MINI PROJECT

Project Type	Mini Project (Team Cybersecurity / Automated Pen-Testing Tool)
Domain	Cybersecurity / Network Security Automation
Repository	https://github.com/pranav-1205/Penetration-Testing-Framework-API
Team	Vedant Paste, Pranav Sadwelkar, Bhavesh Mundye, Tejas Patil
Sponsorship	Academic team mini project no external sponsorship

Abstract

The Penetration Testing Framework API is a web-based security assessment tool that lets users initiate vulnerability scans from a dashboard interface. It automates security assessments using Python and integrates Nmap for network scanning, generating structured security reports.

Problem Statement

Manual penetration testing and network security assessment are time-consuming and require specialised expertise. Small teams and students often lack an accessible, dashboard-driven way to run standard reconnaissance scans and get structured, readable output without manually operating command-line tools.

Objective / Solution

Build a Flask-based REST API that wraps Nmap for network/port scanning and service detection, stores results in a relational schema (users → projects → scans → targets → ports/vulnerabilities), and exposes a web dashboard for initiating scans, viewing results, and (in progress) generating PDF reports.

Key Features

- Network port scanning and service detection via python-nmap
- Automated vulnerability detection
- Dashboard interface for initiating and monitoring scans
- REST API architecture (Flask + Flask-CORS)
- Structured relational schema for users, projects, scans, targets, ports, and vulnerabilities
- PDF report generation (fpdf) in progress

Technology Stack

Technology	Purpose
Python, Flask, Flask-CORS	Backend REST API
python-nmap	Network scanning / security tool integration
HTML, CSS, JavaScript	Frontend dashboard
SQLite	Local relational database

Technology	Purpose
fpdf, Jinja2	Report generation and templating
bcrypt, python-dotenv	Authentication and configuration management

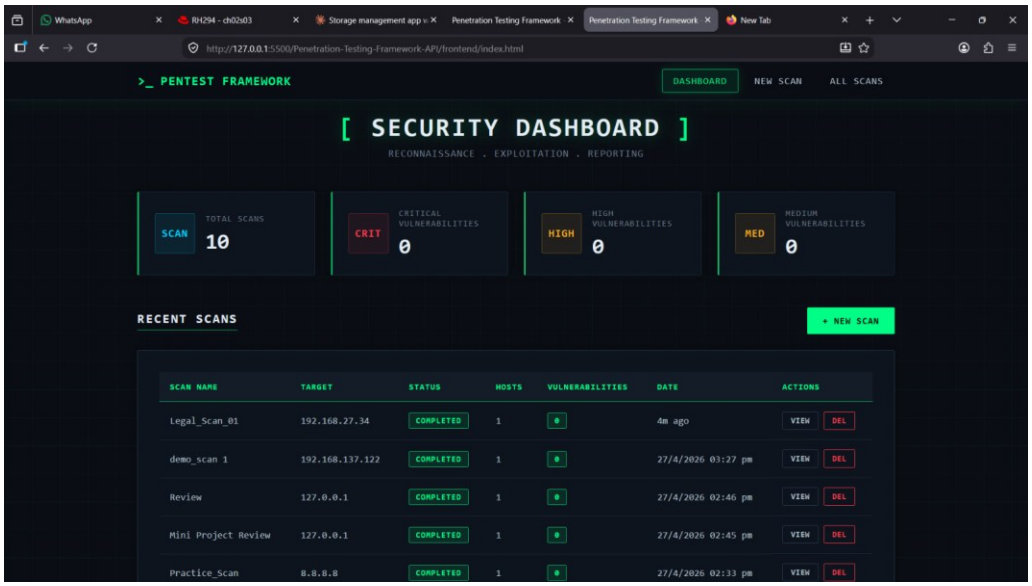
Learning Outcomes

- Hands-on experience with Flask API development
- Understanding of vulnerability scanning concepts
- Practical automation of Nmap from Python
- Secure coding practices
- Report generation and automation

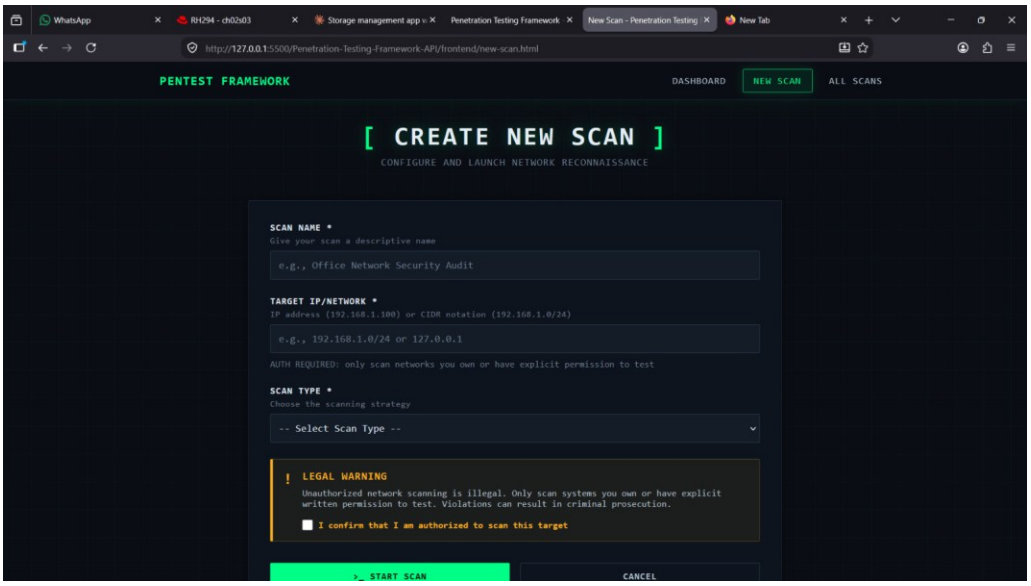
Outcome

A working Flask + Nmap security-scanning dashboard covering port/service scanning and vulnerability logging end-to-end, built and used strictly for educational / authorised-target testing (localhost, VMs, and lab environments), with PDF export and further tool integration in progress.

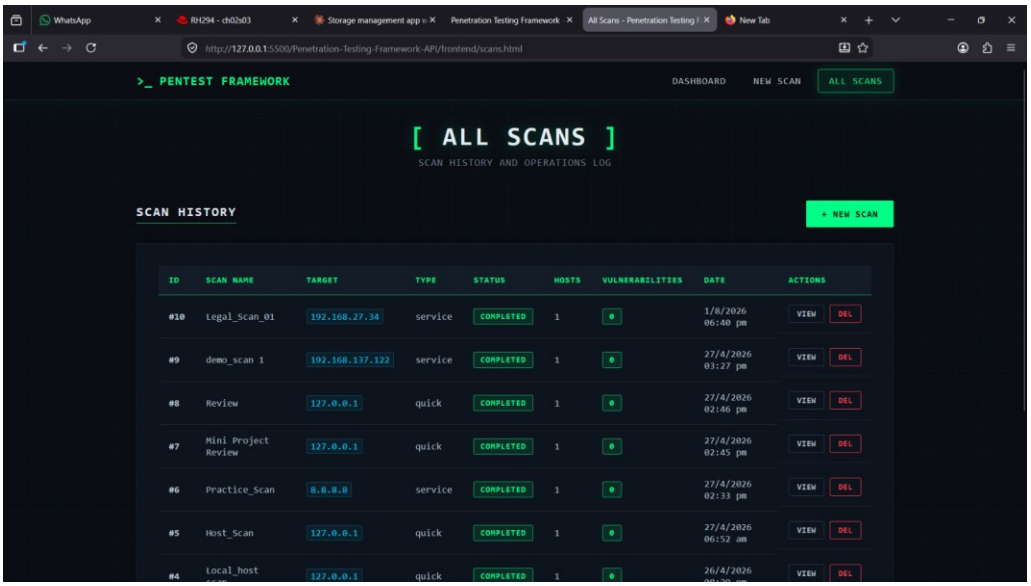
Screenshots



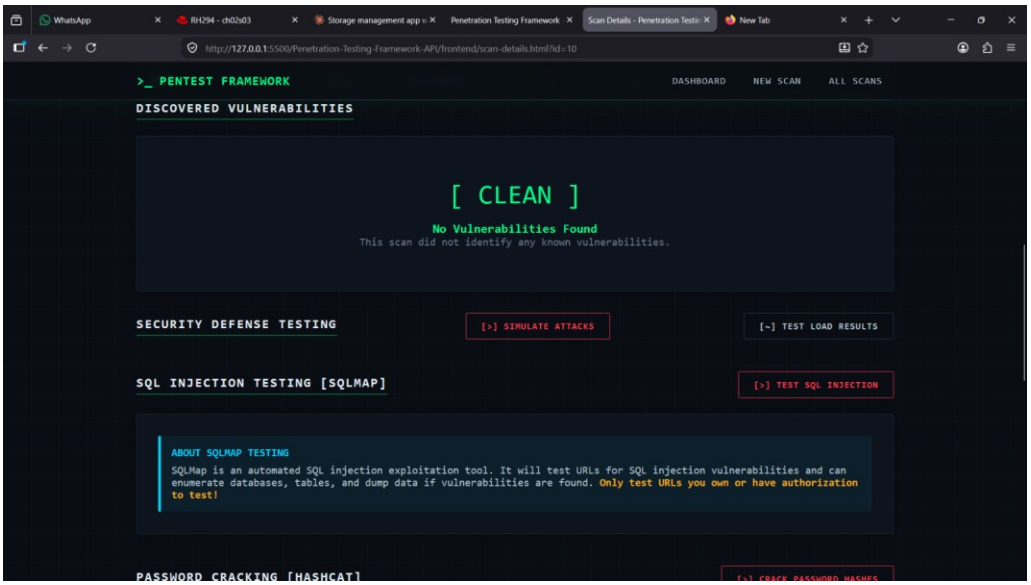
Security dashboard - scan overview and vulnerability summary



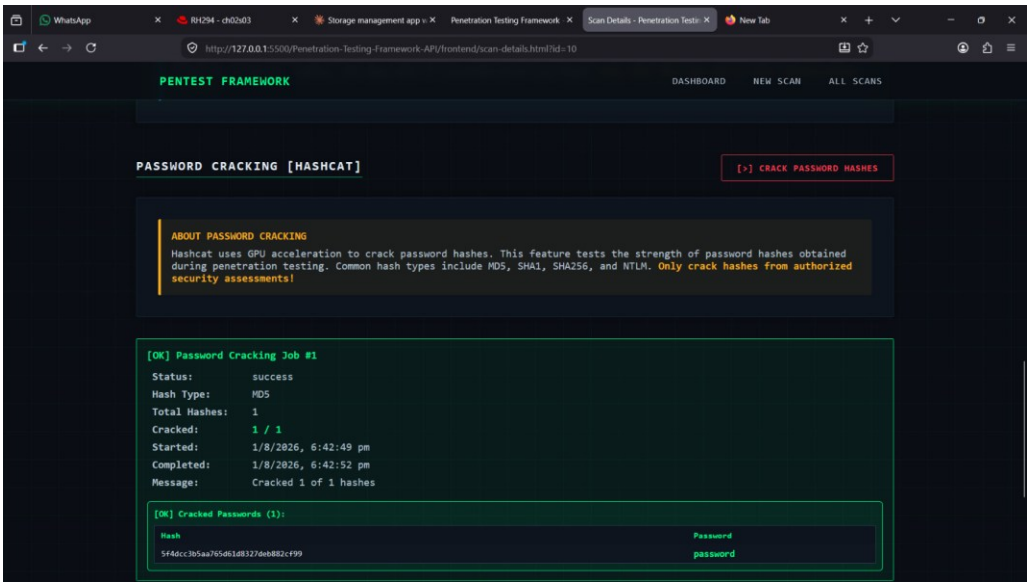
New scan creation - target configuration with legal warning



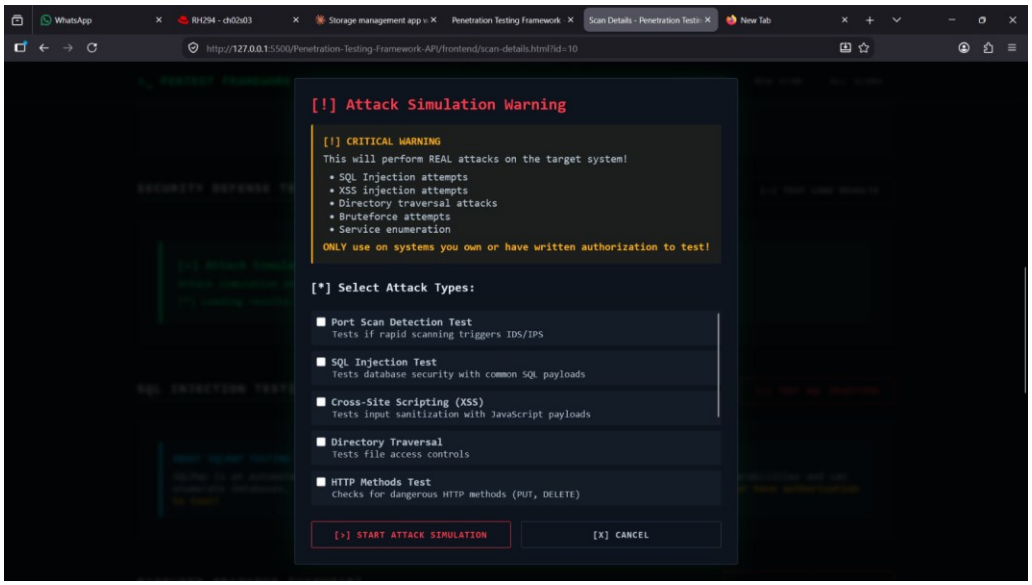
All scans - scan history and operations log



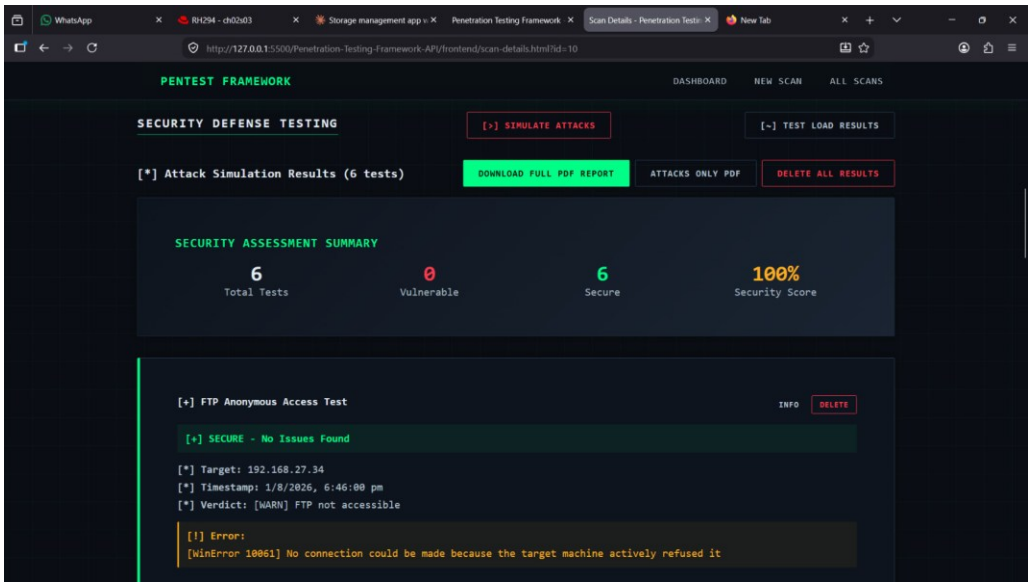
Scan details - discovered vulnerabilities clean, SQL injection testing module



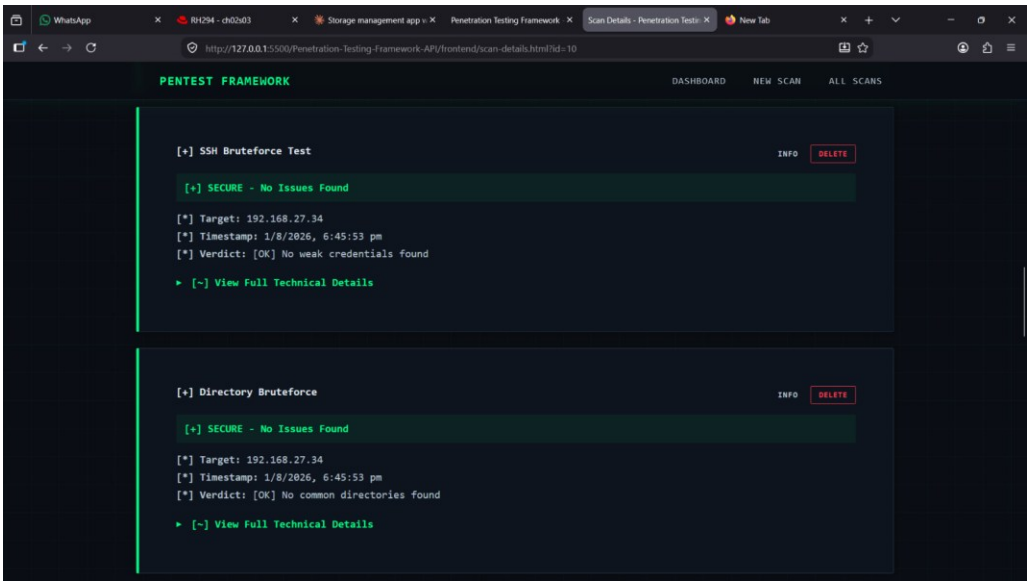
Hashcat password cracking - successfully cracked MD5 hash



Attack simulation warning - selectable attack types



Security defense testing - attack simulation results summary



SSH bruteforce and directory bruteforce test results - secure

6. Collaboration Tool - Real-Time Collaborative Notes Application

(CodTech IT Solutions Internship Task)

MINI PROJECT

Project Type	Mini Project (Internship Task Real-Time Collaboration System)
Company	CodTech IT Solutions
Domain	Software Development
Repository	https://github.com/pranav-1205/WeShare---Collaboration-platform.git
Duration	4 Weeks
Intern ID	CT04DR2516
Mentor	Neela Santosh Kumar
Sponsorship	Completed as part of a structured internship with CodTech IT Solutions

Abstract

This project is a real-time collaborative note-taking web application where users create accounts, own notes, and collaborate on the same document simultaneously via a shared link or QR code. A dashboard lists recent and shared notes, and edits are synchronised instantly across all participants using CRDT-based (Yjs) conflict-free replication over WebSockets.

Problem Statement

Traditional note-taking tools either offer no fine-grained access control (anyone with a link can edit) or don't support true simultaneous multi-user editing without conflicts. There is a need for a secure, account-based way for a group to co-edit a single document in real time, with owner-controlled permissions and no risk of one user's changes overwriting another's.

Objective / Solution

Build an authenticated collaborative editor where a registered user can create a note and share it via a link or QR code, with new collaborators joining as read-only by default. Real-time synchronisation is achieved using Yjs (CRDTs) over Socket.IO, so concurrent edits merge automatically without conflicts, with owner-controlled write permissions, a live people panel, debounced autosave to MongoDB, and dark/light theming.

Key Features

- Account registration and login secured with JWT stored in httpOnly cookies and bcrypt-hashed passwords
- Dashboard with search, recent notes, and notes shared with the user
- Instant note creation with copyable share links and QR-code sharing
- Real-time collaborative editing for multiple simultaneous users via Yjs CRDTs
- Google-Meet-style people panel showing active participants and their roles
- Owner controls: rename the note title, grant or revoke write access per collaborator, and remove users
- Guests and collaborators join as read-only by default until the owner grants write access
- Debounced autosave of note content to MongoDB
- Dark / Light theme toggle with persisted preference

Technology Stack

Technology	Purpose
React (Vite), Quill.js	Frontend UI and rich-text editing
Yjs + y-quill	CRDT-based real-time conflict-free synchronisation
Socket.IO	Real-time client-server communication
Node.js, Express.js	Backend server
MongoDB (Mongoose)	Persisting note content and metadata
jsonwebtoken, bcrypt, cookie-parser	Authentication, password hashing, and httpOnly session cookies
qrcode.react	QR-code note sharing
react-hot-toast	In-app UI notifications

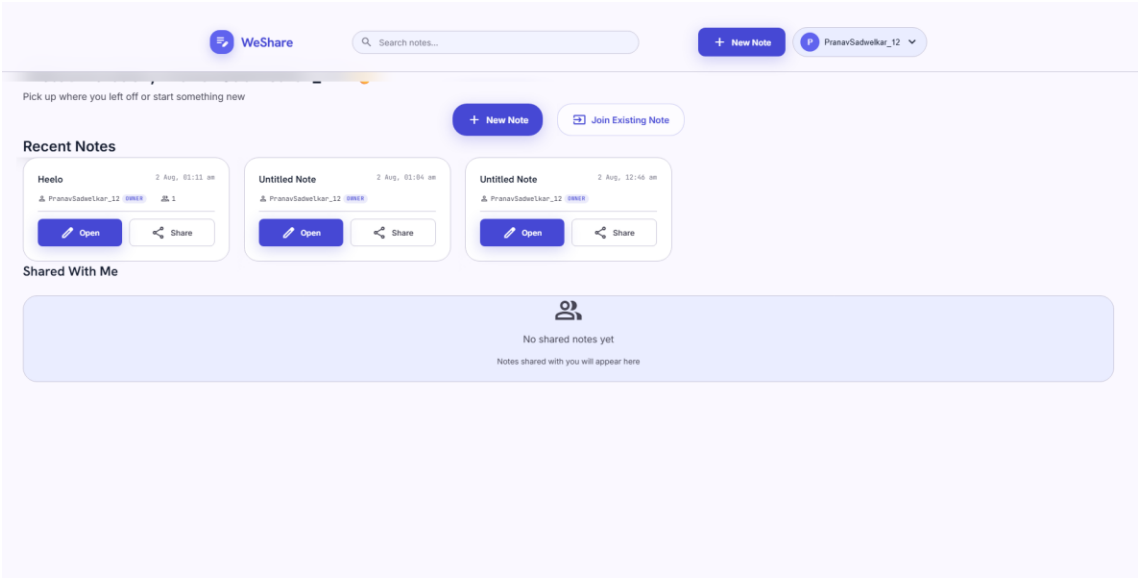
Learning Outcomes

- Real-time systems design using WebSockets and CRDTs
- JWT session management with httpOnly cookies
- Role-based access control in a real-time application (owner / read / write)
- Full-stack integration across React, Express, and MongoDB
- State management under concurrency
- Secure persistence practices: bcrypt password hashing, cookie flags, and server-side authorisation
- Building a clean, custom UI/UX with theming, without a heavy component library

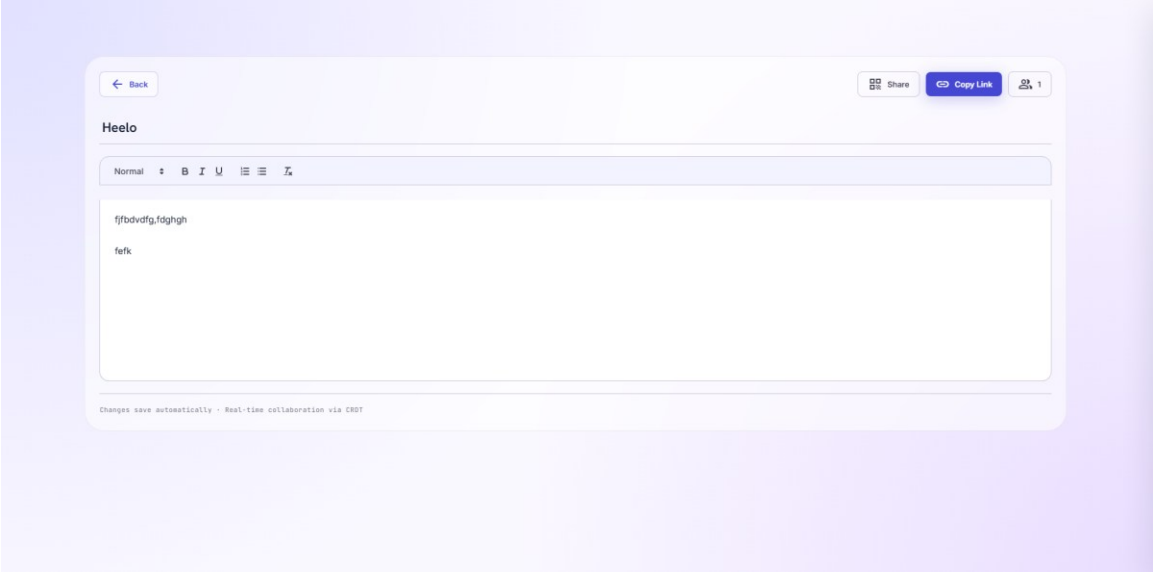
Outcome

A fully functional, secure, Google-Docs-style collaborative notes platform with account-based authentication, owner-controlled permissions, and real-time conflict-free multi-user editing, delivered as the final task of the CodTech IT Solutions internship.

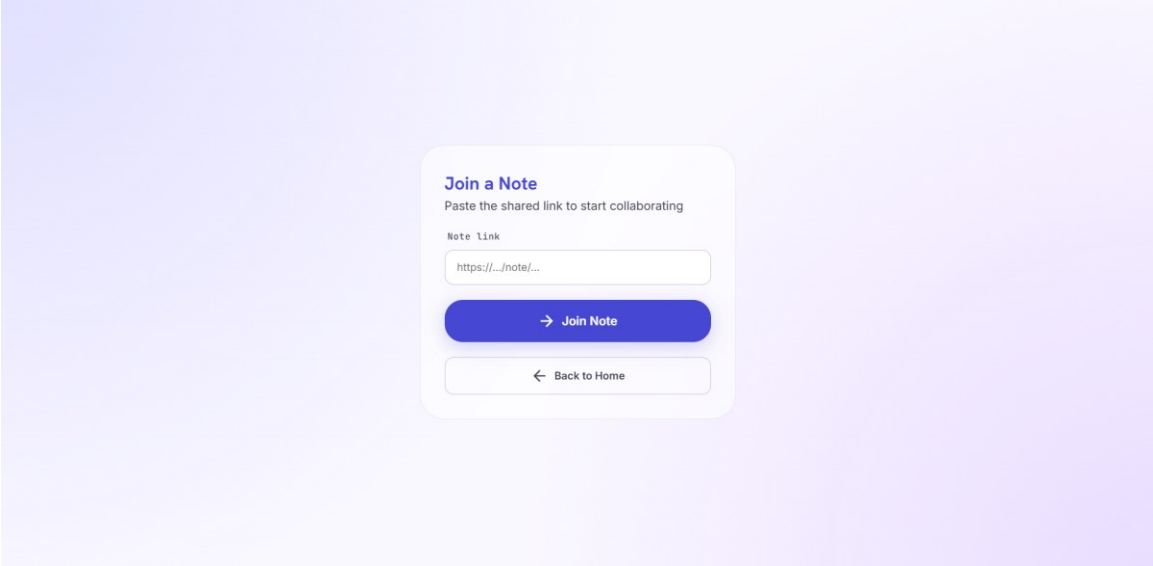
Screenshots



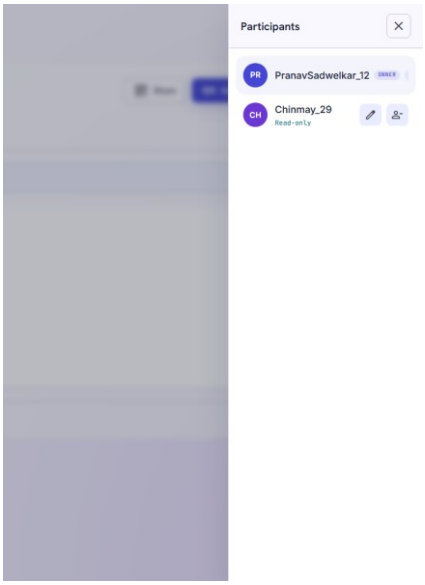
Dashboard - recent notes and join existing note



Real-time editor - autosaving note with share and copy-link controls



Join a Note - joining a shared note via link



People panel - participants with owner and read-only roles